

Lime 创作知识云与产品门槛完整方案

版本：0.2

更新：2026-04-01

用途：用于创始人、产品、设计、研发统一理解 Lime 在开源前提下如何构建长期产品门槛，并明确 Tools / Skills / Search / MCP 的定位、阶段路线图与商业方案

状态：内部方案草稿

一、文档定位

这份文档主要回答六个问题：

- Lime 作为创作产品，应该如何系统性理解 数据 -> 信息 -> 知识 -> 洞察 -> 智慧 -> 影响
- Tools / Skills / Search / MCP 在 Lime 中各自到底扮演什么角色
- 为什么 Lime 不应该对外主打 MCP，但内部仍然应该认真建设连接层
- 在开源前提下，Lime 的真正产品门槛应该建在哪里
- 如果要做类似 Context7 的能力，Lime 应该先做什么、不要先做什么
- 未来 90 天到 12 个月，Lime 应该如何从本地工作台走向“创作知识云”

这不是一份单点功能文档，而是一份：

- 产品战略稿
- 能力分层稿
- 商业门槛稿
- 路线图稿

这里需要先明确一个新的产品边界：

Lime 主软件优先承载浅场景、结构化场景和可回写场景；长篇小说正文创作拆到独立产品。

但这并不意味着底层知识服务要拆成两套。

更合理的做法是：

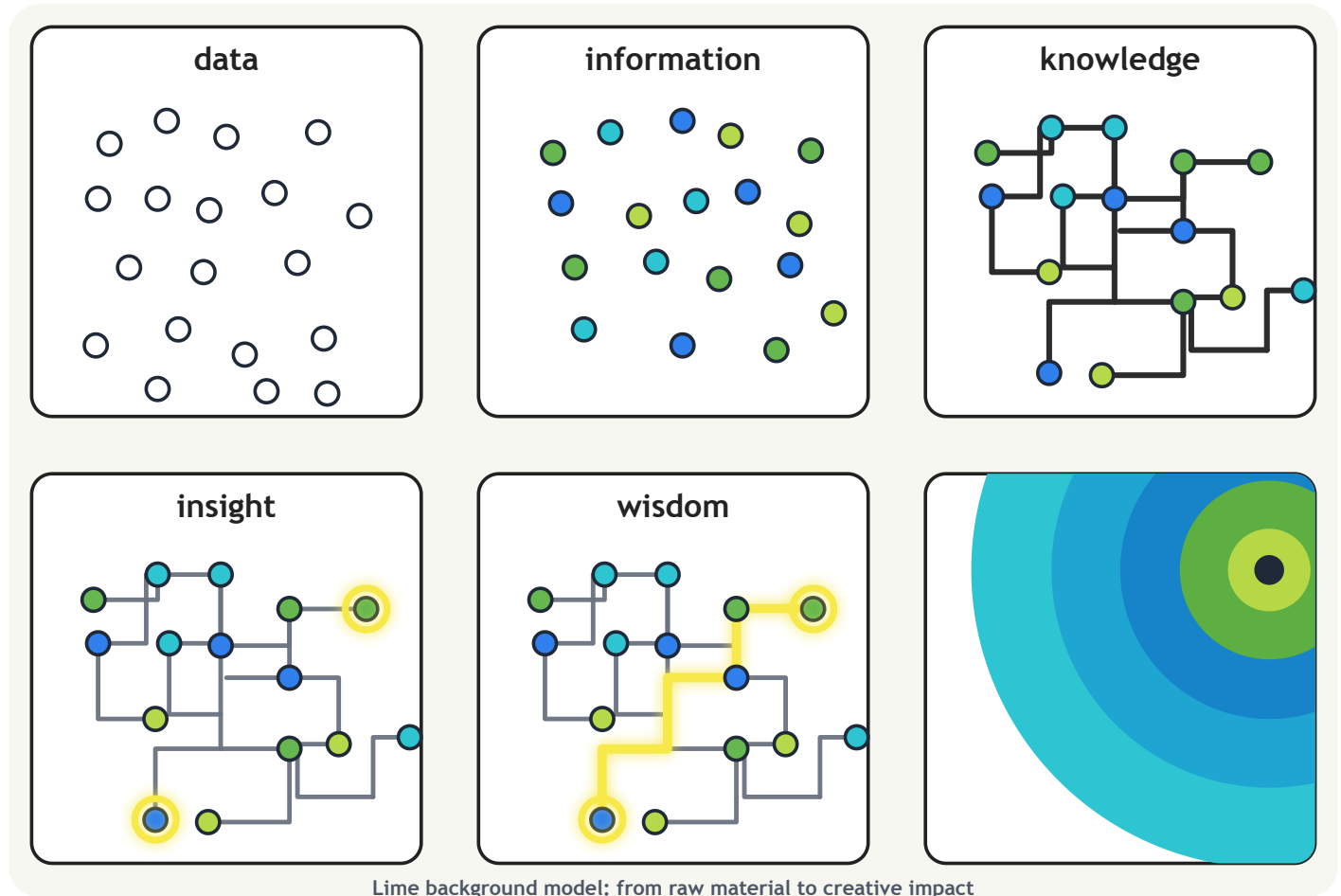
- Lime 主软件复用项目记忆、平台情报、反馈闭环
- 未来独立小说产品也复用同一套知识云与团队资产层
- 前台产品可以分化，底层知识复利应尽量共用

1.1 这份文档的背景图

这份方案的背景，来自一张非常适合解释 Lime 的认知升级图：

Data -> Information -> Knowledge -> Insight -> Wisdom -> Impact

下图为仓库内重绘的 SVG 版本：



这张图本身表达的不是“信息越来越多”，而是“价值如何一步步被组织出来”。

如果拆开来，它包含六层递进关系：

- **Data**
原始、零散、未经组织的事实或素材
- **Information**
被标记、分类、整理后开始具备可读性的内容
- **Knowledge**
信息之间建立关系后形成的结构化认知

- **Insight**
从知识网络中识别出关键问题、关键机会或关键矛盾
- **Wisdom**
基于洞察做出更正确的优先级判断与行动选择
- **Impact**
行动真正作用于外部世界后产生的结果与影响

原图的视觉表达也很有代表性：

- 第一格是散点，表示素材很多但没有组织
- 第二格开始上色，表示信息被分类
- 第三格开始连线，表示知识来自关系，而不是单点事实
- 第四格高亮局部，表示洞察来自识别关键节点
- 第五格高亮路径，表示智慧不是“看见重点”，而是“知道该怎么走”
- 第六格是扩散的同心圆，表示真正重要的是外溢到现实世界的影响

这对 Lime 很重要。

因为 Lime 不能只停留在“帮用户生成更多文本”。

如果 Lime 只是把素材变成更多文本，它最多停留在 **Information** 。

如果 Lime 能把项目素材、作品设定、创作方法、平台知识、发布反馈不断组织起来，它才会逐步进入：

- **Knowledge**
- **Insight**
- **Wisdom**
- **Impact**

因此，后面的所有方案都基于同一个前提：

Lime 的产品目标不是提供更多生成能力，而是帮助创作者把零散素材持续推进为真正有影响力的作品结果。

二、先说结论

2.1 Lime 不该卖 MCP

Lime 不应该把 MCP 作为对外第一层产品叙事。

原因很简单：

- 用户不为协议买单
- 创作者不关心协议名词，只关心“能不能做成、发出、持续变强”
- MCP 更适合作为内部连接标准，而不是外部主卖点

更好的对外讲法是：

Lime 帮创作者把素材变成知识，把知识变成判断，把判断变成作品影响。

2.2 Lime 的门槛不应该建在“工具数量”上

真正的门槛不是：

- 支持多少个 MCP
- 有多少个 Tool
- 有多少个 Search provider

真正的门槛应该是：

- 是否拥有高质量、持续更新的创作知识流
- 是否能把知识稳定变成更好的创作结果
- 是否能形成创作后的反馈闭环

2.3 Lime 的核心路线应该是“创作知识云”

Lime 最值得押注的方向不是做一个“更全的通用 Agent 平台”，而是逐步形成：

本地创作工作台

↓

项目级知识沉淀

↓

托管型创作知识服务

↓

发布与反馈闭环

一句话总结：

Lime 应该开源工作台，开放连接协议，收费于持续更新的创作知识、工作流结果与团队复利资产。

三、用那张图重新理解 Lime

原图表达的是：

Data -> Information -> Knowledge -> Insight -> Wisdom -> Impact

如果放到 Lime 这个“写文章、做研究、做小说相关分析，并为未来独立小说产品沉淀知识”的体系里，它应该被翻译成下面这条价值链。

3.1 Data：原始创作素材

包括：

- 灵感碎片
- 用户输入
- 历史草稿
- 角色设定
- 世界观片段
- 采访记录
- 网页资料
- 平台规则
- 读者评论
- 发布数据

这一层的特点是：

- 零散
- 未归类
- 可用性低
- 噪声高

3.2 Information：被整理过的创作信息

包括：

- 提纲
- 时间线
- 人物关系
- 主题标签
- 平台摘要
- 引用片段
- 来源结论
- 章节摘要
- 设定卡片

这一层的特点是：

- 已经可以被查看和使用
- 但还没有形成完整方法论
- 还不足以稳定指导下一步决策

3.3 Knowledge：可复用的创作知识

包括：

- 作品规则 / canon
- 角色规则
- 世界观词典
- 风格规则
- 类型题材方法论
- 平台改写经验
- 历史案例库
- 团队 Skills
- 项目记忆

这一层开始出现真正的复利。

因为创作者不再每次从零开始，而是开始复用自己的方法和资产。

3.4 Insight：对当前作品问题的判断

例如：

- 主角动机不成立
- 章节冲突不够强
- 标题没有击中目标读者

- 世界观存在设定冲突
- 论点很多，但核心论证不够穿透
- 这篇内容不适合当前平台发布语境

洞察不是“更多信息”，而是：

从复杂信息中识别当前最关键的问题。

3.5 Wisdom：决定下一步最值得做什么

例如：

- 先补角色动机，再更新关系判断
- 先统一叙事视角，再润色文风
- 先做事实核查，再发布
- 先根据平台规则重写开头，再改正文
- 先补一条项目规则，再推进下一步创作

智慧不是“知道很多”，而是：

知道现在最该做什么，不该做什么。

3.6 Impact：作品与创作效率真正发生变化

包括：

- 更容易完成作品或场景任务
- 作品更一致
- 发布更顺畅
- 反馈更好
- 团队方法被持续复用
- 下次创作更快、更准

Impact 是 Lime 的最终价值落点。

Lime 不是为了生成一段文本，而是为了帮助用户形成：

- 作品结果
- 工作流结果
- 资产沉淀结果

四、Lime 能力系统的准确分工

要把 Lime 讲清楚，不能把 Tools / Skills / Search / MCP 当成四个并列按钮。

它们的职责并不在同一个层级。

4.1 Tools：最小动作单元

Tool 回答的是：

“现在具体能执行什么动作？”

典型例子：

- 读文件
- 写草稿
- 改写段落
- 搜网页
- 读取项目设定
- 发布到平台
- 生成封面
- 抓取反馈

Tool 的特点是：

- 粒度小
- 可组合
- 偏执行
- 偏动作

Tool 本身通常不携带完整业务语义。

4.2 Search：外部世界的动态观察能力

Search 回答的是：

“当本地没有答案，或者问题具备时效性时，如何获取外部最新信息？”

它可以包括：

- 通用 WebSearch
- 指定站点搜索

- 平台规则搜索
- 竞品内容搜索
- 垂直知识源搜索

Search 主要服务：

Data -> Information

Search 的核心价值不是“能搜”，而是：

- 新鲜度
- 来源质量
- 时间敏感性
- 交叉验证能力

4.3 Skills：可分发的知识与工作流包

Skill 不是简单的 Prompt，也不是单纯的 Tool 集合。

一个强 Skill 通常同时包含：

- 触发语义
- 领域知识
- 工作流
- 参考资料
- 模板与资产
- 脚本与执行工件
- 验证步骤
- gotchas
- 可选的 hooks 与 memory
- 输出规范
- 质量标准
- 推荐调用面

也就是说，Skill 本质上是：

路由器 + 知识包 + 资产包 + SOP + 验证器 + 输出协议

它回答的是：

“遇到这一类创作任务，应该加载什么知识，用什么模板和脚本，按什么流程推进，如何验证结果，最终交付什么结果？”

这也是为什么一个 Skill 往往天然带有知识库属性。

更重要的是，Skill 不只是“知道什么”，还会影响：

- 怎么触发
- 先做什么
- 哪些坑要避免
- 用哪些脚本和工具
- 如何验证结果是否真的可交付
- 历史执行结果如何被记录与复用

对 Lime 来说，Skill 最重要的价值不只是“自动执行”，而是：

- 沉淀团队方法
- 沉淀团队知识
- 沉淀团队验证方式
- 复用创作经验
- 稳定输出质量
- 把复杂任务变成可重复流程

4.3.1 Anthropic 最佳实践对 Lime 的直接启发

如果参考 Anthropic 在 Claude Code 上的经验，Lime 对 Skill 的理解至少要补上七点。

1. Skill 是目录，不是单一 Markdown

真正强的 Skill 不应该只有说明文档。

它应当有能力同时承载：

- references/
- examples/
- assets/
- scripts/
- templates/
- data/
- 可选 config 与 memory

2. 不要写模型本来就知道的废话

Skill 最有价值的内容，不是重复常识，而是：

- 这个组织独有的做法
- 这个题材独有的知识
- 这个流程最容易踩的坑
- 这个输出最关键的质量判断标准

3. Gotchas 是 Skill 的高信号部分

对 Lime 来说，下面这些都应当进入 Skill 的 gotchas：

- 平台违禁或限流风险
- 设定 continuity 常见冲突
- 常见写崩的人物动机
- 多平台改写时最容易失真的部分
- 生成封面、配图、标题时的低质量模式

4. 要善用渐进披露，而不是一股脑塞进 prompt

Skill 最好的形式不是把所有规则塞进一次上下文，而是：

- 主文档负责路由与触发
- 详细参考按需读取
- 模板按需复制
- 脚本按需调用
- 示例按需查阅

这会直接决定 Skill 的可维护性和模型使用效率。

5. Setup、memory、hooks 不是附属品

很多高价值 Skill 都不是一次性的。

它们会需要：

- 用户配置
- 项目配置
- 历史执行结果
- 只在特定任务中启用的 hooks

对 Lime 来说，这意味着 Skill 应该能逐步从“静态能力包”成长为“长期项目协作者”。

6. Skill 必须包含验证，不然容易停留在“看起来不错”

对于创作类产品，验证不只是测试代码。

它同样包括：

- 发布前检查
- 设定一致性检查
- 结构完整性检查
- 平台适配检查
- 风格守恒检查

7. Skill 需要分发、度量和淘汰机制

好的 Skill 体系不是越多越好，而是：

- 可发现
- 可安装
- 可度量
- 可迭代
- 可淘汰重复或低价值 Skill

4.3.2 适合 Lime 的 Skill 分类

结合 Anthropic 的分类方式，Lime 更适合重点建设以下几类 Skill。

1. Canon / 风格 / 平台参考型

对应：

- 作品设定
- 人物规则
- 题材知识
- 风格参考
- 平台写作规则

2. 验证型

对应：

- 章节 continuity 检查
- 发布前检查

- 多平台版本核对
- 风格偏移诊断

3. 数据获取与分析型

对应：

- 平台规则抓取
- 热点与竞品分析
- 评论聚类
- 发布表现分析

4. 工作流自动化型

对应：

- 从选题到主稿
- 从主稿到多平台改写
- 从章节到时间线更新
- 从发布到反馈回流

5. 模板与脚手架型

对应：

- 小说章纲模板
- 品牌内容模板
- 平台标题模板
- 可复制的输出结构

6. 质量与审校型

对应：

- 结构审查
- 风格审查
- 引用审查
- 平台风险审查

因此，Lime 的 Skill 战略不应该停留在“做一些提示词包”。

更准确的说法是：

Lime 要建设的是一个可分发、可验证、可迭代的创作知识与工作流生态。

4.4 MCP：连接协议，而不是产品主语

MCP 更准确的定位是：

连接层中的开放协议实现。

它回答的是：

“**Lime** 如何把外部知识源、资源、工具、提示和工作区边界，以标准方式接进来？”

MCP 内部可能包括：

- tools
- resources
- prompts
- roots

但对产品来说，MCP 不应该被讲成“用户价值本身”。

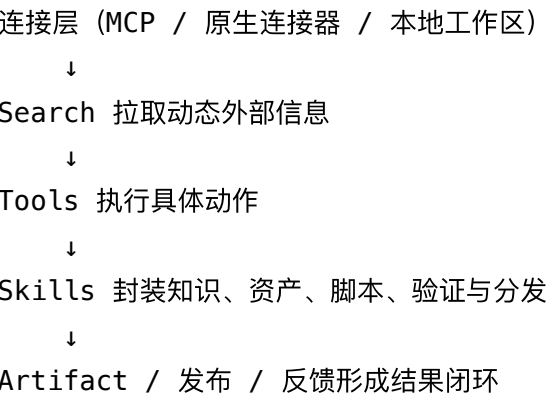
因为协议只是通路，不是结果。

所以更合理的产品表达应该是：

- 对外讲：连接你的资料、知识源和工作流
- 对内讲：优先用 MCP 与原生连接器实现标准接入

4.5 四者的关系

把它们串起来，可以得到一条更准确的链路：



因此：

- Tool 是动作
- Search 是观察
- Skill 是知识与工作流包
- MCP 是连接协议

五、Lime 应该如何组织成一个好产品

一个好产品，不是把所有能力暴露给用户，而是把能力组织成对用户自然的工作流。

5.1 前台按创作意图组织，不按技术名词组织

用户不应该先看到：

- MCP
- Tool
- Search provider
- Runtime Binding

如果是 Lime 主软件前台，用户应该先看到：

- 调研
- 构思
- 拆书分析
- 改写
- 校对
- 发布
- 复盘

这意味着前台的第一层入口必须是“创作任务”，而不是“技术原语”。

5.2 后台按职责分层组织

Lime 更合理的后台结构应是：

感知层

输入 / 文件 / 导入 / 网页 / 搜索 / 反馈采集

知识层

项目规则 / 风格 / 项目记忆 / 题材知识 / 平台知识 / Skills

判断层

诊断 / 规划 / 风险识别 / 改写策略 / 发布建议

执行层

读 / 写 / 改 / 查 / 发 / 同步 / 生成 / 提取

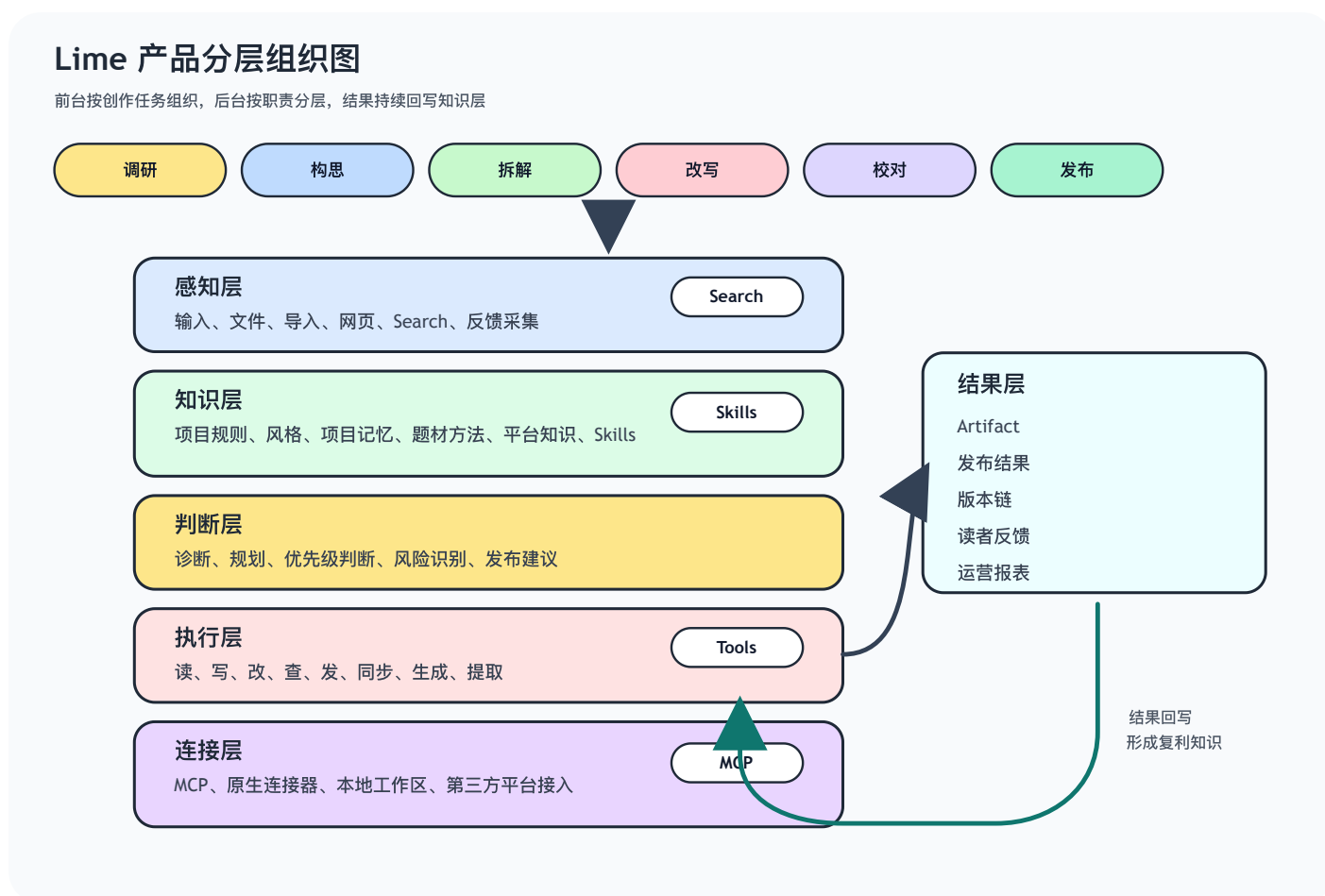
连接层

MCP / 原生连接器 / 本地工作区 / 第三方平台接入

结果层

Artifact / 发布结果 / 版本链 / 读者反馈 / 运营报表

下图展示了推荐的产品组织方式：



5.3 用户体验的关键原则

Lime 的体验设计应遵守下面四条：

原则 1：先给结果，不先给配置

不要让创作者先配置一堆 provider、search、server、tool。

应该先让用户拿到：

- 第一版提纲
- 一组摘要或结构卡
- 一次设定诊断
- 一次平台改写结果

原则 2：先有默认工作流，再有高级自定义

Lime 不要一开始就把自己讲成“任意编排平台”。

更应该先提供几个打透的默认链路：

- 选题调研 -> 主稿 -> 配图 -> 多平台改写
- 导入章节 -> 章节摘要 -> 人物卡 -> 关系线 -> 主题提炼 -> 结论沉淀

长篇小说正文创作那条更深的工作流，应在独立产品承载，但可以复用同一套知识层、平台情报和反馈闭环。

原则 3：每次创作都要回写知识层

如果每次写完都只是生成文本，Lime 就永远停留在“聊天助手”。

Lime 必须把高价值结果回写为：

- 角色卡
- 时间线
- 设定词典
- 风格规则
- 平台模板
- 发布表现

原则 4：所有能力最终都要服务 Impact

Impact 不是“生成成功”，而是：

- 创作更稳定
- 发布更有效
- 团队更容易复用方法
- 用户更愿意长期留下项目和资产

六、开源产品如何建立门槛

Lime 是开源的，但开源不等于没有门槛。

关键是：

把协议和客户端开源，把持续更新的知识与结果服务做成门槛。

6.1 适合开源的部分

建议长期开源：

- 桌面客户端
- 本地工作区
- 基础 Tool runtime
- 基础 Skill runtime
- 本地文件索引
- 本地项目知识组织
- 基础连接能力
- 基础 Search 接口层

这些部分开源的好处是：

- 容易获得开发者采纳
- 易形成生态
- 降低用户试用门槛
- 有利于本地优先产品心智

6.2 应该建立商业门槛的部分

建议重点做成托管服务或高级版能力：

- 高质量托管知识库
- 持续更新的数据管道

- 垂直题材知识服务
- 平台规则与内容策略情报
- 多源搜索聚合、排序、去重、可信度评估
- 版本化引用与来源追踪
- 团队级共享 Skills 与知识资产
- 发布数据回流与效果分析
- 反馈闭环优化

这类能力的共同特征是：

- 持续维护成本高
- 数据更新有运营壁垒
- 用户很难自己做得同样好
- 一旦积累起来复利非常强

6.3 最健康的商业结构

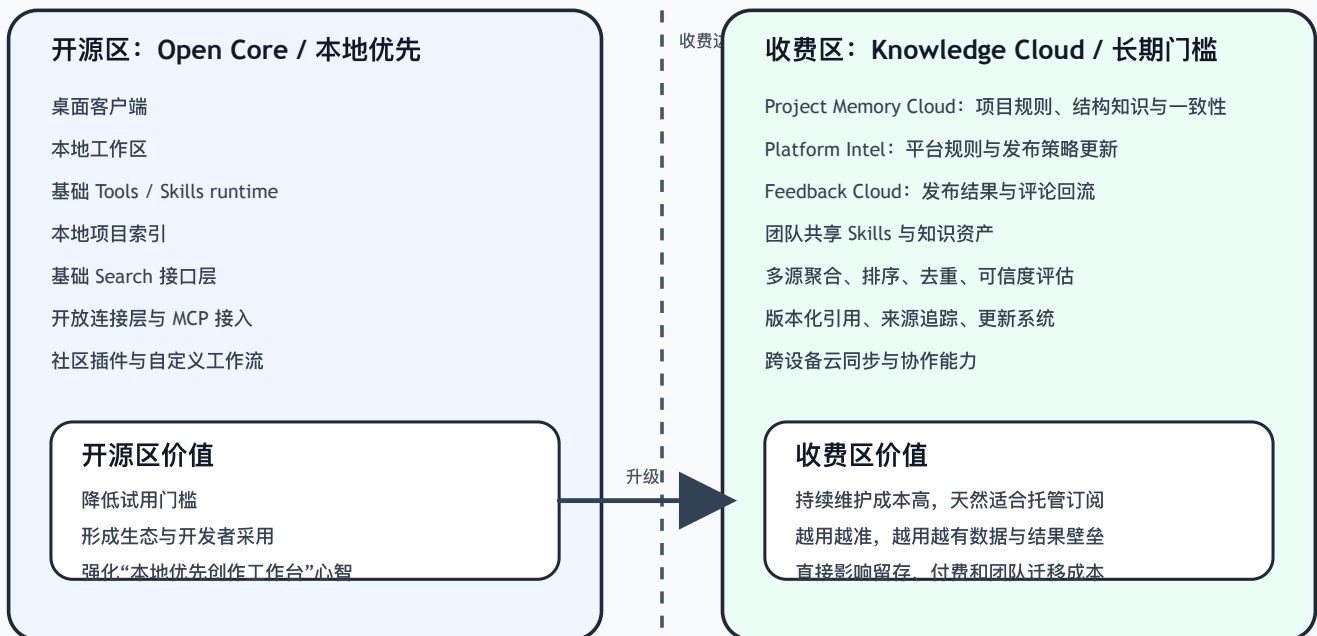
建议采用：

开源客户端 + 开放连接层 + 托管型创作知识云

下图更直观地表示了开源区与门槛区的边界：

Lime 开源与商业门槛边界图

原则：开源基础能力，收费于持续更新的项目记忆、结果服务与团队复利资产



不是卖“更多 MCP”，而是卖“更好的结果、项目记忆与持续复利”

也就是：

- 协议开放
- 客户端开放
- 核心知识流与反馈闭环收费

这样既符合开源产品精神，也能保护商业利益。

七、Lime 不要先做什么

为了建立真正的门槛，有几件事不应成为第一优先级。

7.1 不要先做“通用搜索大全”

原因：

- 同质化严重
- 用户感知价值低
- 成本高

- 很容易被替代

如果 Lime 的主卖点变成“我们也有 WebSearch”，那几乎没有壁垒。

7.2 不要把 MCP 数量当卖点

“支持 100 个 MCP server”不是强护城河。

因为这更像功能表，而不是用户结果。

7.3 不要先做泛 Agent 平台

如果 Lime 过早把自己抽象成通用 Agent 平台，就会稀释创作主战场。

更好的做法是：

- 先打透创作
- 再向外延展

八、最值得做的知识连接 / Search 方向

如果参考 Context7 的启发，Lime 应该学到的不是“做一个 MCP”，而是：

做一个高可信、可引用、持续更新、对创作任务真正有价值的知识流。

这些能力既可以通过 MCP 暴露，也可以通过原生连接器或托管服务实现。

下面是更适合 Lime 的五个方向。

8.1 Project Memory / Canon Service

这是最应该优先做的方向。

价值

服务文章、研究、拆书分析、系列内容，以及未来独立小说产品的项目创作。

核心提供：

- 人物卡检索

- 项目规则检索
- 历史章节与历史文章检索
- 时间线与关系线检索
- 主题块与结论卡检索
- 已发布版本检索
- 项目术语词典检索

用户价值

帮助创作者解决：

- 前后设定打架
- 长文跨章遗忘
- 多人协作时口径漂移
- 系列内容风格不统一

门槛来源

- 项目越写越深
- 索引越积越厚
- 规则越用越准
- 换平台难度高

战略判断

这是最符合 Lime 核心定位、最不容易被通用搜索替代的方向。

8.2 Platform Intel Service

服务内容发布与增长。

核心提供：

- 平台规则
- 标题策略
- 排版样式
- 敏感风险
- 当前有效内容样式
- 平台差异建议

适用平台例如：

- 小红书
- 公众号
- 知乎
- 抖音图文
- Substack
- Amazon KDP

门槛来源

- 更新频率高
- 需要持续维护
- 需要真实内容样本
- 对运营结果有直接影响

8.3 Genre Lore Service

服务垂直题材创作。

例如：

- 网文题材库
- 历史题材资料
- 神话系统
- 悬疑结构
- 叙事母题库
- 人物原型库

它的意义是：

不是让用户“搜互联网”，而是让用户“搜被整理过、适合创作使用的高价值知识”。

8.4 Style Reference Service

服务风格稳定与风格迁移。

核心提供：

- 作者风格拆解

- 品牌语气
- 句式偏好
- 节奏习惯
- 正反例对照
- 风格守恒建议

8.5 Feedback Loop Service

服务真正的结果复利。

核心提供：

- 发布后表现
- 评论聚类
- 收藏、转化与停留
- 哪类标题更强
- 哪类开头掉读更严重
- 哪类章节最受欢迎

这个方向的重要性在于：

它可以把 Impact 重新回灌为下一轮 Data 和 Knowledge 。

九、推荐优先级

9.1 优先级排序

建议优先级如下：

1. Project Memory / Canon Service
2. Platform Intel Service
3. Feedback Loop Service
4. Style Reference Service
5. Genre Lore Service

9.2 为什么先做 Project Memory / Canon Service

因为它同时满足以下条件：

- 与 Lime 创作定位最一致
- 最贴近日常主链
- 最容易形成用户留存
- 最容易形成项目复利
- 最不容易被通用搜索替代
- 最容易与本地工作区、Artifact、Skill 形成闭环

一句话判断：

先做“作者自己的项目大脑”，再做“外部世界搜索”。

十、Lime 创作知识云的产品方案

为了让上述方向形成一个真正可卖的产品，建议把服务端产品统一收敛为：

10.1 Lime Knowledge Cloud

它不是一个“工具商城”，而是一个面向创作者和内容团队的托管知识与结果服务层。

建议包括四个子产品：

1. Project Memory Cloud

解决：

- 项目记忆
- 作品设定与项目规则
- 角色、关系和时间线一致性
- 系列内容与结构化场景的一致性

它既服务 Lime 主软件，也可以作为未来独立小说产品的底层知识服务。

2. Platform Intel Cloud

解决：

- 平台策略
- 发布改写

- 标题与结构适配
- 风险检查

3. Team Skill Cloud

解决：

- 团队共享 Skills
- 参考资料、模板、脚本与资产打包
- 标准输出格式与验证规则
- Skill 分发、安装、版本管理
- Skill 使用度量与优胜劣汰

4. Feedback Cloud

解决：

- 发布结果回流
- 评论与读者信号
- 下轮创作优化建议

这四层组合起来，才是 Lime 的长期门槛。

十一、为什么这套方案符合你的利益

因为它遵守了一个对开源创业者最重要的原则：

开源基础能力，收费于持续更新、持续维护、持续产生结果的服务。

11.1 用户买的不是锁定，而是效果

如果收费点是：

- 私有格式
- 封闭协议
- 阉割本地功能

用户会强烈抵触。

但如果收费点是：

- 托管知识更新
- 平台策略更新
- 结果反馈回流
- 团队资产共享
- 高质量引用与可信排序

用户会更容易接受。

11.2 你获得的是可持续门槛

这类门槛不是一次性写出来的，而是持续积累出来的：

- 数据壁垒
- 排序壁垒
- 更新壁垒
- workflow 壁垒
- 团队协作壁垒

11.3 它不和开源精神冲突

因为开源用户仍然可以：

- 用本地工作台
- 用本地 Skill
- 用基础 Tool
- 接自己的数据源

而付费用户买的是：

- 更省时间
- 更准
- 更懂项目
- 更懂平台
- 更有结果闭环

十二、技术实现建议

为了避免路线过重，建议按“最小可用知识服务”来做。

12.1 统一远程能力接口

托管型知识服务建议先提供最小的远程能力面：

- search
- fetch
- resolve
- cite

含义分别是：

- search：找到候选知识项
- fetch：取回完整内容或片段
- resolve：将模糊提问映射到项目实体
- cite：返回可引用来源与版本信息

不要一开始暴露太多花哨接口。

12.2 统一索引对象模型

无论是 Canon、Platform 还是 Feedback，都尽量统一为带元数据的文档对象：

- id
- project_id
- doc_type
- title
- summary
- source
- version
- updated_at
- tags
- entities
- citations
- confidence

这样后续：

- Search
- Skill
- 引用
- 结果回写

都会更容易统一。

12.3 统一写回通道

Lime 不能只有“读知识”的能力，还必须有“写回知识”的能力。

建议设计标准回写通道：

- 保存角色卡
- 保存设定变更
- 保存章节摘要
- 保存平台表现
- 保存风格偏好
- 保存反馈洞察

12.4 统一评分与引用体系

高质量知识服务必须回答四个问题：

- 这条信息来自哪里
- 它有多新
- 它可信度多高
- 为什么这条结果排在前面

没有这四条，知识服务就很容易退化成普通检索。

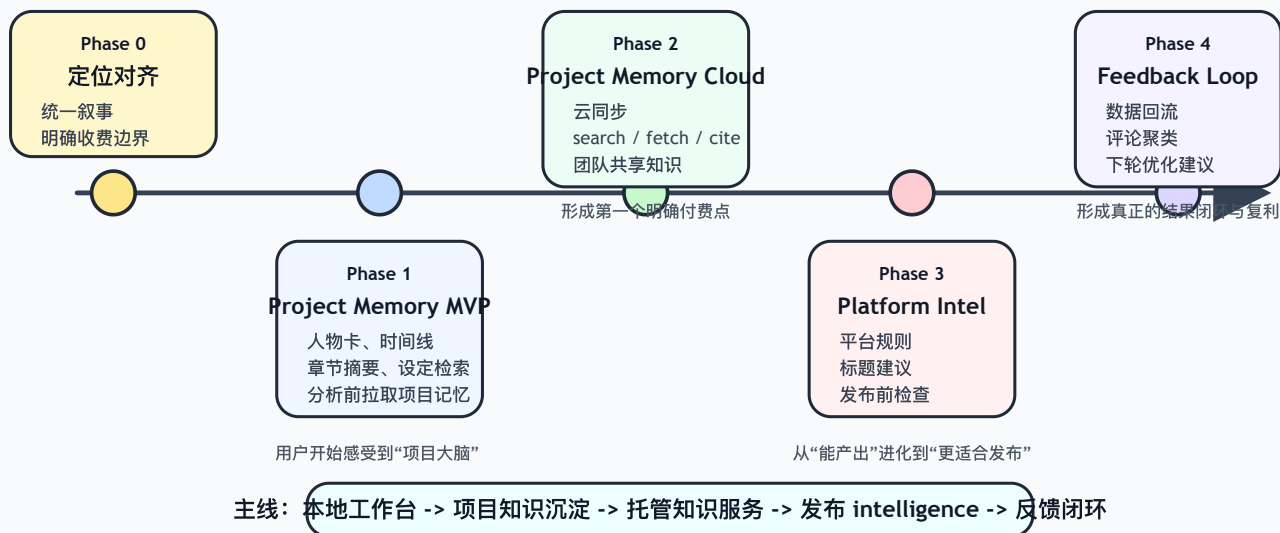
十三、阶段路线图

下面给出一个更可执行的阶段方案。

先看整体路线图：

Lime Knowledge Cloud 阶段路线图

从定位对齐到项目大脑、云同步、平台 intelligence、反馈闭环的渐进路径



13.1 Phase 0：定位对齐（1-2 周）

目标：

- 统一内部叙事
- 确定第一优先知识产品
- 确定开源与收费边界

产出：

- 本文档定稿
- Lime 对外一句话定位
- 第一阶段产品范围表
- 远程知识接口草案

13.2 Phase 1：Project Memory MVP（3-6 周）

目标：

- 让 Lime 真正具备“项目大脑”

最小能力：

- 项目内文档索引

- 角色卡索引
- 时间线与关系线索索引
- 章节摘要索引
- 设定冲突搜索
- 分析或改写前自动拉取相关项目记忆

用户可感知结果：

- 系列内容和结构化浅场景明显更稳
- 多篇文章 / 多章节分析之间一致性更强
- 不容易忘设定、关系和项目规则

对未来独立小说产品而言，这一步也能直接复用为更强的项目大脑。

13.3 Phase 2: Project Memory Cloud 托管版（4-8 周）

目标：

- 从本地索引升级到托管知识服务

最小能力：

- 项目云同步
- 版本化文档实体
- 远程 search/fetch/cite
- 团队共享项目知识
- 多设备接续

商业价值：

- 形成首个明确付费点

13.4 Phase 3: Platform Intel（4-8 周）

目标：

- 让 Lime 从“能产出”进化到“更适合发布”

最小能力：

- 平台规则库

- 标题建议
- 发布前检查
- 多平台改写建议
- 风险提醒

用户可感知结果：

- 发布更省心
- 不同平台适配更稳定

13.5 Phase 4: Feedback Loop (6-10 周)

目标：

- 让 Lime 真正形成结果闭环

最小能力：

- 发布后数据回流
- 评论聚类
- 高表现内容模式抽取
- 下一轮改写建议

用户可感知结果：

- Lime 不只是会生成，而是会越来越会产出更好的结果

十四、商业化建议

14.1 版本策略

建议至少区分三层：

Free

- 本地工作台
- 基础 Skills
- 基础 Tools
- 本地项目索引

- 基础 WebSearch 接入

Pro

- Project Memory Cloud
- 基础 Platform Intel
- 云端项目同步
- 高级引用与知识回溯
- 高级 Skill 包与托管分发

Team

- 团队共享 Skills
- 团队知识库
- 协作项目
- 发布与反馈回流
- 团队资产治理
- Skill 使用度量与治理

14.2 收费理由必须围绕结果

不要主打：

- 更多协议
- 更多 provider
- 更多 server 数量

应该主打：

- 项目一致性
- 更稳的结构化分析与改写质量
- 更高效的发布适配
- 团队知识复用
- 更好的内容结果

十五、成功指标

不要只看功能完成数，要看这几个指标。

15.1 创作留存

- 同一项目 7 日回访率
- 同一项目 30 日持续创作率

15.2 知识沉淀深度

- 每个项目的角色卡数量
- 每个项目的摘要覆盖率
- 被回写到项目记忆的内容比例

15.3 Skills 复用率

- 同一 Skill 的重复使用率
- 团队共享 Skill 的采用率

15.4 发布结果改善

- 多平台改写使用率
- 发布前检查使用率
- 标题建议采纳率

15.5 付费转化理由

重点验证用户是否因为以下理由愿意付费：

- 项目一致性
- 团队共享知识
- 平台规则更新
- 反馈回流

十六、主要风险与应对

风险 1：做成“搜索工具集合”

后果：

- 用户感知弱

- 壁垒弱
- 难形成付费理由

应对：

- 以任务结果组织产品
- 以知识闭环而不是工具数量为导向

风险 2：过早做成重平台

后果：

- 工程复杂度高
- 用户价值不集中
- 产品叙事发散

应对：

- 先打透一个主链
- 先做 Project Memory，再扩展

风险 3：知识服务质量不足

后果：

- 回答看似丰富，实际不可信
- 用户不敢依赖

应对：

- 强制版本信息
- 强制来源引用
- 强制更新时间
- 强制可信度排序

风险 4：开源和商业边界不清

后果：

- 社区不满
- 付费价值模糊
- 团队反复摇摆

应对：

- 开源基础能力
- 商业化集中在托管知识与结果服务
- 文档和版本策略提前说清楚

十七、未来 90 天建议

如果只看未来 90 天，建议只做下面五件事。

1. 统一 Lime 的一句话定位

建议统一为：

Lime 是一个面向创作者与内容团队的本地优先 AI 创作工作台，帮助用户把素材沉淀为知识，把知识推进为可发布作品。

2. 明确对外不主打 MCP

对外第一屏不讲：

- MCP
- Tool 数量
- Search provider 数量

对外第一屏只讲：

- 项目一致性
- 团队方法复用
- 从草稿到发布

3. 启动 Project Memory MVP

这是最重要的第一刀。

4. 设计 Project Memory Cloud 的托管边界

明确哪些能力：

- 本地免费
- 云端收费

5. 为 Platform Intel 留好接口，但不先铺太大

先做连接点和数据模型，不急着做大而全平台情报系统。

十八、最终判断

Lime 如果只是一个开源创作客户端，那么它的门槛会很有限。

Lime 如果只是一个支持 MCP、Tools、Search、Skills 的工作台，那么它的叙事会太技术化。

Lime 真正值得做成的，是一个：

以本地创作为入口，以项目知识沉淀为核心，以托管型创作知识云为门槛，以发布与反馈闭环为长期复利的创作操作系统。

其中：

- Tools 负责动作
- Search 负责观察外部世界
- Skills 负责承载领域知识与方法
- MCP 负责作为连接层中的标准协议

但这些都只是中间层。

Lime 最终要卖的，不是协议，不是工具，不是连接数量。

Lime 最终要卖的是：

创作者持续产出、持续发布、持续变强的能力。